



Customer Management System Development

Practical Skills Module 6: Topic 1

[Visit our website](#)

Practical Skills Module 6: Topic 1

Topic Code	PM-06-PS01
Topic	Python exercises may be conducted at the end of the skills programme or may be integrated into learning at the end of the applicable module/section

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, connected to framework/ environment of own choice, the learner must be able to:

- PA0101 Use DML commands with Python
- PA0102 Understand Python Data Types
- PA0103 Use Python's Predefined String Functions
- PA0104 Master Python key concepts
- PA0105 Manipulate strings and data
- PA0106 Create and execute custom functions
- PA0107 Create, sort and modify Python lists
- PA0108 Deal with error handling in Python
- PA0109 Convert decimals

Applied Knowledge

- AK0101 Concept, definition and functions of Python programming principles

Internal Assessment Criteria

- IAC0101 Expected results are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-06



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Customer Management

You will create a Customer Management System that includes database operations. Steps 1 and 8 focus on working with a SQLite database, creating, manipulating, and handling errors when accessing customer data. The other steps cover general Python tasks like data validation, string manipulation, functions, and list operations, which can be completed independently of the database.

Note: To keep your code organized and easy to test, implement each step as a separate function within the same file. You can use a main function to call these step functions in sequence.

Create a Python file called **customer_management.py** and complete the following:

Step 1: Use DML (Data Manipulation Language) commands with Python

- Create a SQLite database and a table for storing customer information (name, balance, email).
- Add customer records for the following individuals:

Name	Balance	Email
Alice Smith	R 120.50	Alice@example.com
Bob Johnston	R 75.30	Bob@example.com

- Implement the following operations:
 - **Create:** Insert new customer records into the database.
 - **Read:** Retrieve and display customer information from the database.
 - **Update:** Modify an existing customer's information (e.g., change the balance).

- **Delete:** Remove a customer record from the database.

Step 2: Understand Python Data Types

- Validate customer data types (strings, floats, etc.):
 - Declare sample variables (e.g. `name = "Alice Smith", balance = 120.50`).
 - Use a conditional to check if the name is a string and if the balance is a float.

Tip: You can use the `isinstance()` function to validate the data types, which will return `True` if the value matches the specified type.

Step 3: Use Python's Predefined String Functions

- Declare variables to hold the raw customer name and email (e.g., `"alice smith"`, `"ALICE@EXAMPLE.COM"`).
- Format this data for consistency using Python's built-in string methods:
 - Use `.strip()` to remove any leading or trailing whitespace.
 - Use `.title()` to ensure proper capitalization of names (e.g., `"alice smith" → "Alice Smith"`).
 - Use `.lower()` to standardize email addresses (e.g., `"Alice@Example.Com" → "alice@example.com"`).

Step 4: Master Python key concepts

- Use loops, conditionals, and logical operations to manage customer balances:
 - Define a list of customer records, where each customer is represented as a dictionary with keys like `"name"` and `"balance"`.
 - Loop through the list using a `for` loop.
 - Use an `if` statement to check if a customer's balance is below a specific threshold (e.g., `R 100.00`).
 - Print a message for each customer that meets the condition.

Step 5: Manipulate strings and data

- Extract and display specific information about customers:

- Define a string with customer name, email, and balance separated by commas.
- Use the `.split()` string method to separate the values into individual components.
- Convert the balance from a string to a float.
- Print each piece of information in a readable format.

Step 6: Create and execute custom functions

- Create a function to calculate a discount for customers based on their balance.
- Define a function called `calculate_discount()` that accepts a customer's balance as a parameter.
- Use a conditional to check if the balance is greater than 100.
 - If it is, return a 10% discount based on the balance.
 - Otherwise, return 0 (no discount).
- Call the function with a sample balance and print the result.

Step 7: Create, sort and modify Python lists

- Create a list of dictionaries, where each dictionary contains a customer's name and balance.
- Use the `sorted()` function to sort the list by the customer's balance.
- Print the sorted list of customers, displaying both their name and balance.

Step 8: Deal with error handling in Python

- Handle errors when retrieving customer data from a database by using try-except blocks.
- For example, check if the query result is empty and raise an error if the customer is not found, then catch and display the error message gracefully.

Step 9: Convert decimals

- Convert balances to display them as integers.
- Format the balance as a currency using an f-string to show two decimal places (e.g., \$120.75).

Tip: You can import Python's `math` module to convert a decimal balance (float) to an integer using `math.floor()`.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
